

Machine Learning — Part 2

Study Notes: Regression Algorithms & Linear Regression

These notes organize and explain the concepts covered in the lecture on regression algorithms, with a focus on **Simple & Multiple Linear Regression**: the mathematical intuition (best-fit line, cost function, gradient descent), model evaluation (R^2 / Adjusted R^2), overfitting & underfitting, regularization (Ridge/Lasso), and the end-to-end coding workflow (train/test split → fit → predict → evaluate) used in the two hands-on projects (insurance charges & car price prediction).

How to use this document: Each section builds on the previous one. Work through it in order — the mathematical intuition (Sections 3–5) is the foundation for understanding why the code in Section 8 is written the way it is.

1. Overview: Regression vs. Classification

Supervised learning problems fall into two broad categories, depending on what you are trying to predict:

- **Regression algorithms** — used when the target variable is **continuous** (can take many different numeric values), e.g. predicting *salary* from *age*, or *house price* from house features.
- **Classification algorithms** — used when the target variable is **categorical / discrete**, e.g. predicting whether a passenger *survived or not* (0/1) on the Titanic.

This lecture (Part 2 of the Machine Learning series) focuses entirely on **regression**. Classification (logistic regression, KNN, etc.) is covered in Part 3, and unsupervised learning afterwards.

2. What Is Regression?

Regression is a way to predict a number. It helps us understand how one variable changes as another variable changes — i.e., it finds the relationship between an **input** and an **output**.

Intuitive example (lemonade stand): Suppose you track how many glasses of lemonade you sell each day and the temperature that day. After a while you notice: the hotter the day, the more lemonade you sell. Regression is the math that lets you predict *exactly how much more* lemonade you'll sell as the temperature rises — e.g., predicting how many glasses you need to stock if the temperature crosses 50°C.

Core idea: **Regression finds patterns among numbers so we can make smart, quantitative predictions.**

3. Simple Linear Regression

3.1 Problem Statement

Consider a dataset with one **input variable** (feature), Age, and one **output variable** (target), Salary. The goal: given a new, unseen Age, predict the corresponding Salary.

By convention:

- The **input variable** is plotted on the **X-axis**.
- The **output variable** (the thing being predicted) is plotted on the **Y-axis**.

3.2 Scatter Plot & the Best Fit Line

Plotting every (Age, Salary) pair gives a **scatter plot**. Linear Regression then tries to draw a single straight line through the middle of these points — called the **Best Fit Line** — such that it can be used to predict the output (Y) for *any* input (X), including values that were never seen in the original data (e.g., predicting salary for age = 29 even though 29 was not in the dataset).

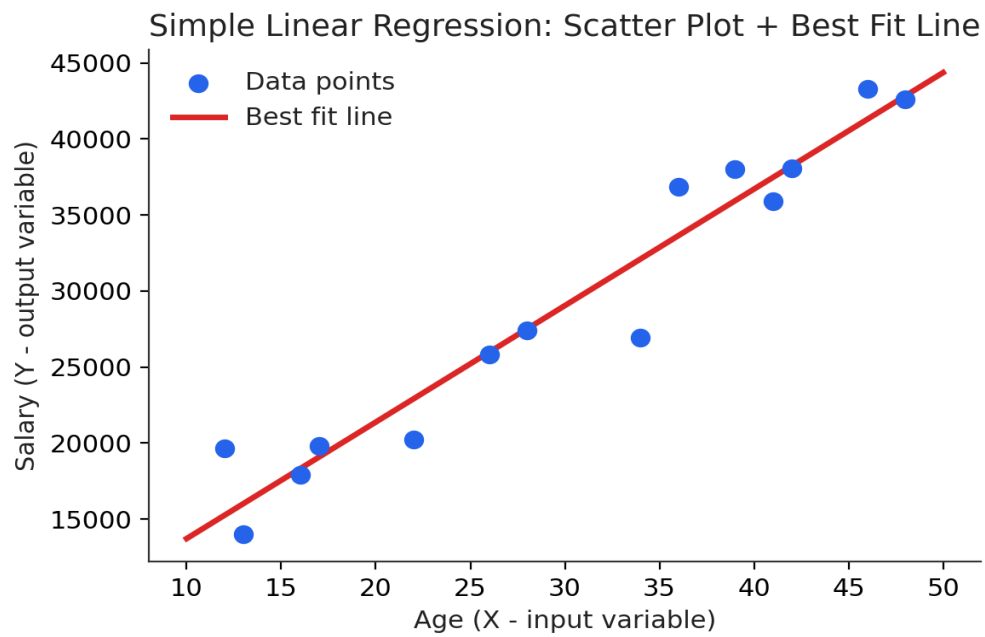


Fig 1. A scatter plot of Age vs. Salary with the Best Fit Line drawn through it. New predictions are made by reading the line's Y-value at the given X.

The key question the rest of this section answers: **out of the infinitely many possible straight lines you could draw through a scatter plot, how do you mathematically choose the single best one?**

3.3 The Line Equation: $y = mx + b$

Every candidate straight line can be written as:

$$y = m \cdot x + b$$

Where:

- **m** — the **slope**: how much y changes for one unit increase in x .
- **b** — the **intercept**: the value of y when $x = 0$ (where the line crosses the Y-axis).
- **x** — the data point you want to predict for.
- **y** — the model's prediction.

Worked example: if $m = 1$ and $b = 0$, then for $x = 1.5$, $y = 1 \times 1.5 + 0 = 1.5$. If instead $m = 5$ and $b = 0$, the same $x = 1.5$ now gives $y = 5 \times 1.5 + 0 = 7.5$ — a completely different line and a completely different prediction. This shows that changing m and b changes which line gets drawn, and hence changes every prediction made from it.

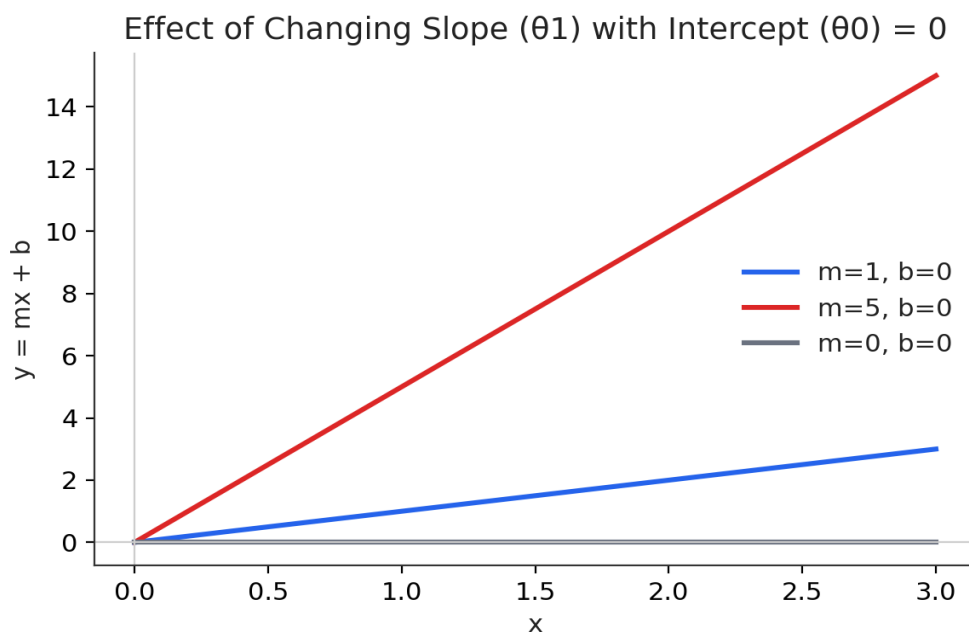


Fig 2. Different values of slope (m) produce different lines through the same intercept — larger $m \Rightarrow$ steeper line.

3.4 Hypothesis Notation (θ notation)

In the machine-learning literature this same equation is usually written using **theta** (θ) notation instead of m and b — this is exactly the same formula, just relabeled:

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$

- θ_0 (theta-zero) corresponds to **b**, the intercept.
- θ_1 (theta-one) corresponds to **m**, the slope.
- $h_{\theta}(x)$ is the model's **prediction** (often written $\hat{y}$).

The central problem of Linear Regression is: **find the perfect values of θ_0 and θ_1** that make the line fit the data best.

4. Residual Error & the Cost Function

4.1 Residual Error

For every data point, the **residual error** is the gap between the **actual value** (y) and the **value predicted by the line** ($\hat{y} = h_{\theta}(x)$):

$$\text{residual} = h_{\theta}(x_i) - y_i$$

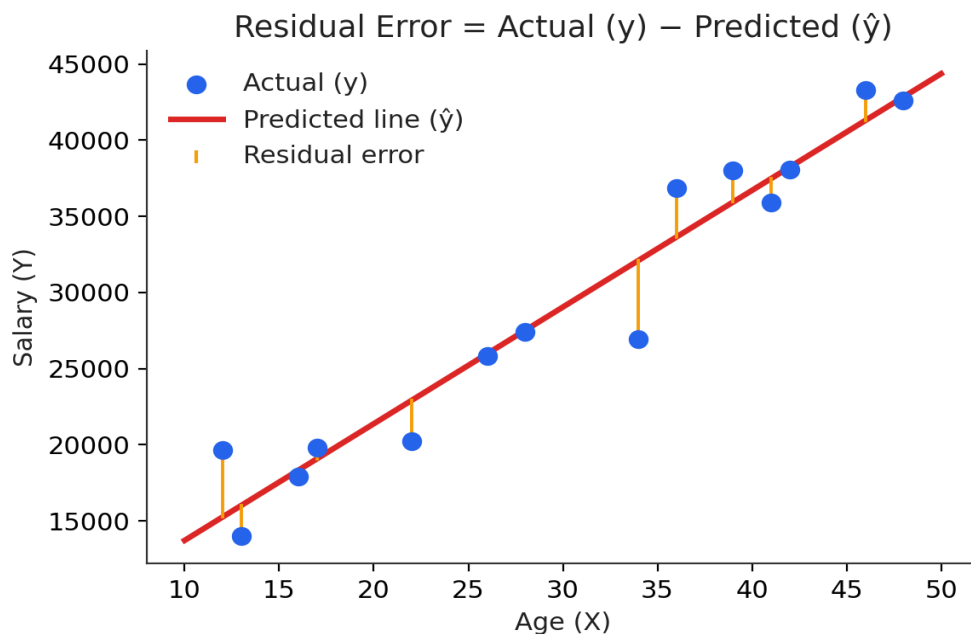


Fig 3. Residual error for each point is the vertical distance between the actual point and the line's predicted value at that x .

A line that passes very close to most points has small residuals; a poorly-fitting line has large residuals. This is exactly the yardstick used to compare candidate lines.

4.2 Cost Function (Mean Squared Error)

To judge how good an entire line is (not just one point), sum up the squared residual for every data point, then average it. Squaring is used so negative and positive differences don't cancel out. This is the **Cost Function**, denoted J :

$$J(\theta_0, \theta_1) = (1 / 2m) \cdot \sum_{i=1}^m [h_{\theta}(x_i) - y_i]^2$$

Where m here is the **number of training examples** (rows), and the extra factor of 2 in the denominator is a convenience that makes the derivative used later in gradient descent cleaner.

Every choice of (θ_0, θ_1) draws a different line — and every different line produces a different cost $J(\theta_0, \theta_1)$.
Different lines $\Rightarrow$ different residuals $\Rightarrow$ different cost.

Goal restated mathematically: find the (θ_0, θ_1) pair that **minimizes** $J(\theta_0, \theta_1)$.

5. Gradient Descent

5.1 From Cost to a Curve

If you fix θ_0 and vary θ_1 across many values, plotting θ_1 (x-axis) against the resulting cost $J(\theta_1)$ (y-axis) traces out a bowl-shaped curve. The point at the very bottom of this bowl is called the **Global Minima** — this is the θ_1 value that produces the lowest possible cost, i.e. the best line.

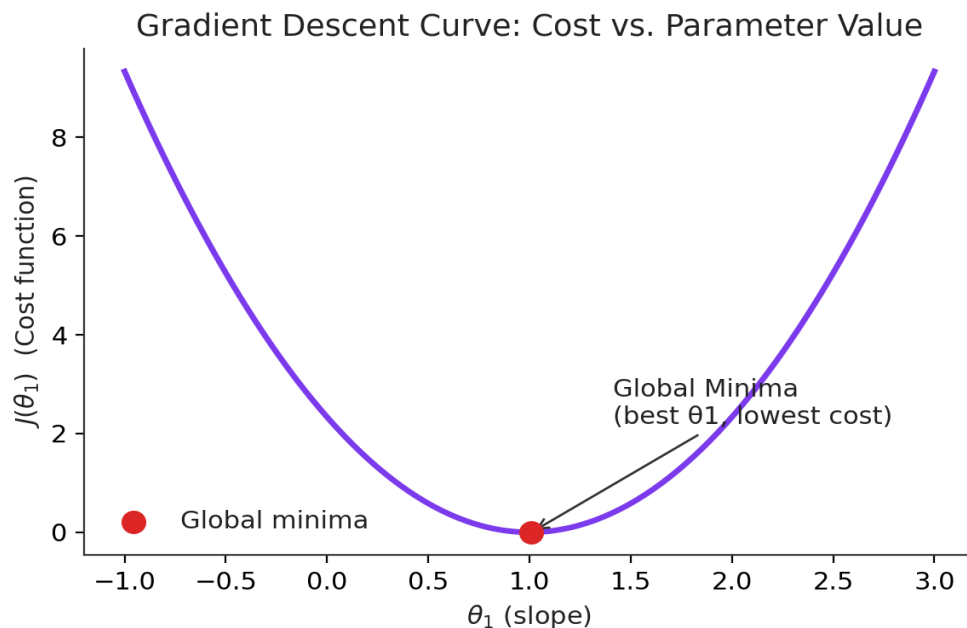


Fig 4. The Gradient Descent curve: plotting cost J against different values of the parameter θ_1 . The lowest point is the Global Minima.

When both θ_0 and θ_1 are allowed to vary together (as in the real Multiple/Simple Linear Regression case), this curve becomes a 3-D bowl-shaped surface, and the same idea applies: we are searching for the lowest point on that surface.

5.2 The Repeat Convergence Algorithm

Gradient Descent is the iterative algorithm that **walks down** this curve/surface step by step until it reaches (or gets very close to) the global minima. It repeatedly updates θ_0 and θ_1 in the direction that reduces the cost, using the **Repeat Convergence Theorem** update rule, until convergence.

5.3 Learning Rate (α)

Each step's size is controlled by a hyperparameter called the **learning rate**, denoted α (**alpha**):

- If α is **too high**, the algorithm takes huge jumps, can overshoot the minimum repeatedly, and may never converge.
- If α is **too low**, the algorithm converges reliably but slowly, taking small, steady steps toward the global minima.

Effect of Learning Rate (α) on Gradient Descent

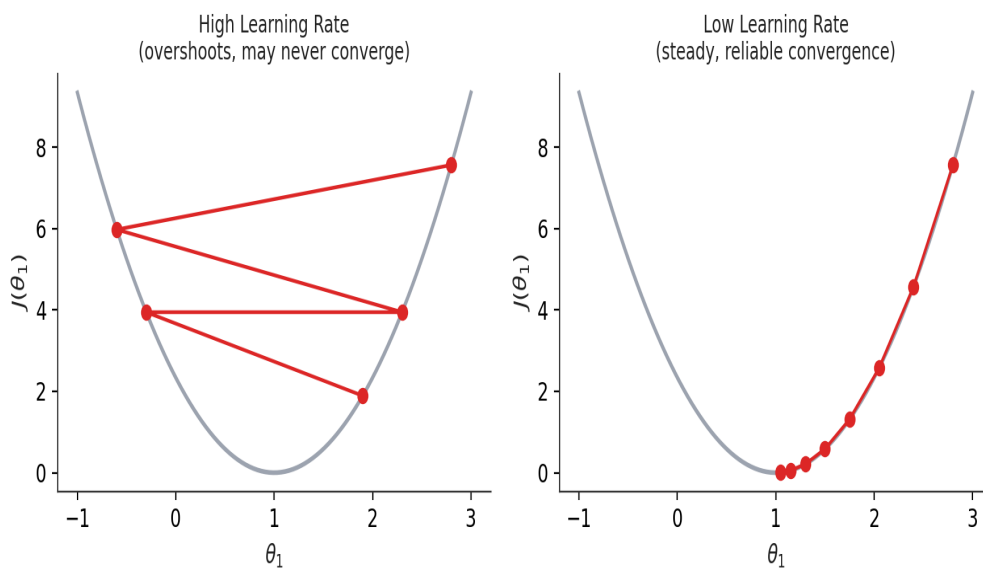


Fig 5. A high learning rate overshoots and oscillates; a low learning rate steadily approaches the global minima.

A small, well-tuned learning rate is generally preferred over a large one, because reliably (even if slowly) reaching the true minimum beats bouncing around it indefinitely.

6. Multiple Linear Regression & the Hyperplane

Simple Linear Regression uses only **one input feature** (e.g., Age only). In real datasets you almost always have **multiple input features** (e.g., age, BMI, number of children, smoker status, region, etc., when predicting insurance charges). The hypothesis equation simply extends to include one θ term per feature:

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 + \dots + \theta_n x_n$$

With one input feature, the "best fit line" is a straight 2-D line. With **two or more** input features, the "best fit line" generalizes into a flat **plane** (or higher-dimensional equivalent) that is fitted through the data cloud — this is called a **hyperplane**.

Multiple Linear Regression: Hyperplane

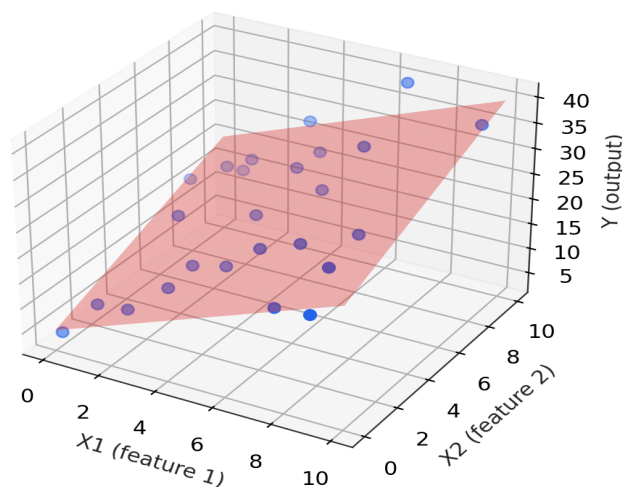


Fig 6. With two input features (X1, X2), the model fits a flat hyperplane through the 3-D data cloud instead of a single line.

Because features often have very different scales and some are categorical text values (e.g. "male"/"female", "smoker"/"non-smoker"), Multiple Linear Regression pipelines typically need:

- **Encoding** — converting categorical text columns into numbers (Label Encoding or One-Hot Encoding).
- **Feature scaling** — standardizing/normalizing numeric columns (e.g. with StandardScaler) so that no single feature dominates purely because of its scale.

7. The End-to-End Modeling Workflow

End-to-End Model Building Workflow

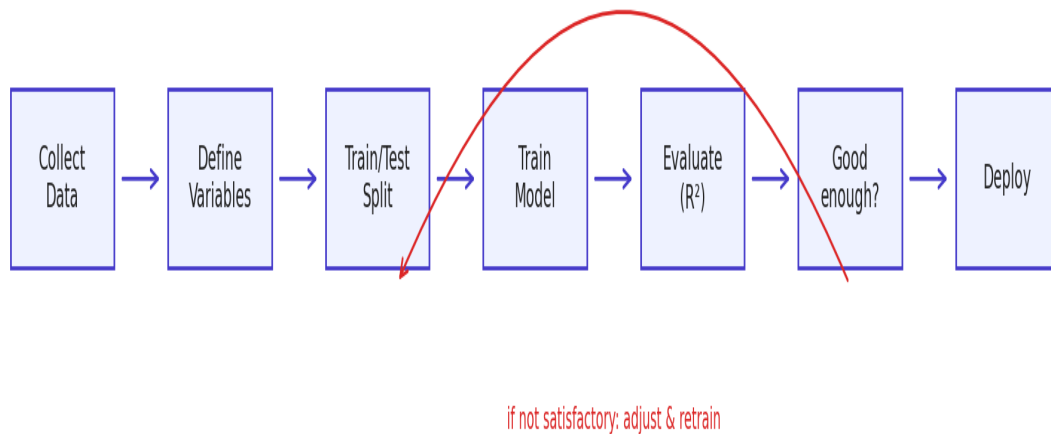


Fig 7. The general workflow followed for any supervised learning model, including the linear regression projects built in this lecture.

The general recipe used throughout the lecture (and applicable to almost any ML model):

- **1. Collect data and define input (X) / output (y) variables.**
- **2. Preprocess** — clean missing values, encode categorical columns, scale numeric columns.
- **3. Train/Test Split** — typically 80% training data / 20% testing data.
- **4. Train the model** on the training split only.
- **5. Predict** on the held-out test split.
- **6. Evaluate** predictions against the real test values (R^2 , Adjusted R^2 , etc.).
- **7. Decide** — if performance is satisfactory, the model is ready; if not, go back, adjust features/hyperparameters, and retrain.

8. Implementing Linear Regression in Python (scikit-learn)

This is the code skeleton used for both hands-on projects (insurance charges prediction and car price prediction). Comments explain what each line accomplishes conceptually.

```
# 1. Separate input features (X) and target/output (y)
X = final_df.drop(columns=['charges'], axis=1)
y = final_df['charges']

# 2. (If needed) Encode categorical columns
X_encoded = pd.get_dummies(X, columns=['model', 'transmission', 'fuel_type'],
                             drop_first=True).astype(int) # One-Hot Encoding

# -- OR --
from sklearn.preprocessing import LabelEncoder
encoder = LabelEncoder()
for col in categorical_columns:
    X_label[col] = LabelEncoder().fit_transform(X_label[col])

# 3. Feature scaling for numeric columns
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_encoded[num_cols] = scaler.fit_transform(X_encoded[num_cols])

# 4. Train / Test split (commonly 80% / 20%)
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X_encoded, y, test_size=0.20, random_state=42)

# 5. Create & train the model
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(X_train, y_train)

# 6. Predict on the unseen test set
y_predictions = model.predict(X_test)

# 7. Evaluate the model
from sklearn.metrics import r2_score
r2 = r2_score(y_test, y_predictions)

n = X_test.shape[0] # number of rows (test samples)
p = X_test.shape[1] # number of features (columns)
adjusted_r2 = 1 - (1 - r2) * (n - 1) / (n - p - 1)
```

Why train/test split? The model must never be evaluated on the same rows it was trained on — that would not tell us how it performs on genuinely new, unseen data. The 20% "test" portion stands in for real-world unseen data during evaluation.

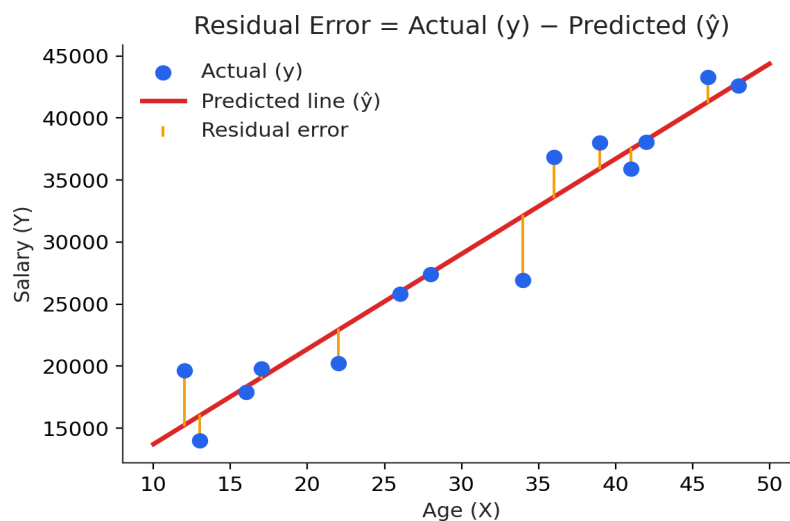
9. Model Evaluation: R^2 and Adjusted R^2

9.1 R^2 (Coefficient of Determination)

R^2 measures **how good the model is** — the proportion of variance in the output that the model is able to explain, compared to simply always predicting the mean of y .

$$R^2 = 1 - (\sum (y_i - \hat{y}_i)^2) / (\sum (y_i - \bar{y})^2)$$

The numerator (**Sum of Residuals**) is the total squared error of the actual best-fit line's predictions. The denominator (**Sum of Totals**) is the total squared error if you had instead just drawn a flat line at the mean of y ($\bar{y}$) and used that as your "prediction" every time.



The numerator of R^2 is built from exactly this kind of residual, but measured against the fitted line rather than the mean.

- R^2 is always between 0 and 1 (as a fraction) — a well-fitting model has a **small** numerator relative to the denominator, pushing R^2 **close to 1**.
- An R^2 of 0.86 (86%) means the model explains 86% of the variability in the target — generally considered a strong result for real-world data.

9.2 Why Adjusted R^2 Is Needed

A subtle problem with plain R^2 : **adding more features to a model can never decrease R^2** , even if the new feature is **completely unrelated** to the target (e.g. adding "hours of sleep" when predicting salary). R^2 will still creep up slightly, misleadingly suggesting the bigger model is better.

Adjusted R^2 corrects for this by penalizing the score for every extra feature added, so that only features which meaningfully improve the fit are rewarded:

$$\text{Adjusted } R^2 = 1 - [(1 - R^2) \cdot (n - 1) / (n - p - 1)]$$

- **n** — number of data points (rows) used.
- **p** — number of input features (columns/predictors) used.

Diagnostic rule of thumb: if adding a new feature makes R^2 go up but makes Adjusted R^2 go **down**, that feature is not actually helping the model — remove it.

10. Overfitting & Underfitting

A trained model can fail in two opposite ways when it meets new (test) data:

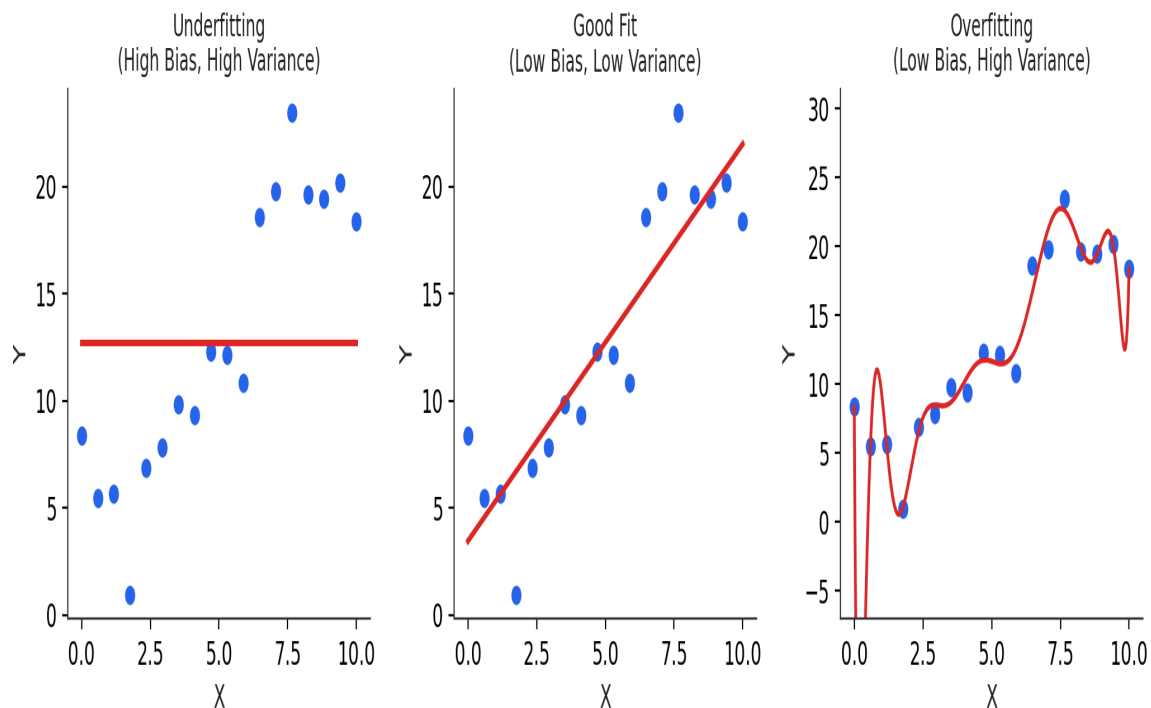


Fig 8. Underfitting (too simple, misses the pattern), a good fit (captures the real trend), and overfitting (too complex, memorizes noise).

Condition	Training performance	Testing performance	Bias / Variance
Overfitting	Very good	Poor	Low Bias, High Variance
Good fit (ideal)	Good	Good	Low Bias, Low Variance
Underfitting	Poor	Poor	High Bias, High Variance

Overfitting means the model has essentially memorized the training data (including its noise), so it performs great on training rows but poorly on new rows. **Underfitting** means the model is too simple to capture the real underlying trend, so it performs poorly everywhere. The ideal model performs consistently well on both training and testing data.

11. Regularization: Ridge & Lasso Regression

When a model is overfitting or underfitting, **Ridge** and **Lasso Regression** can help by adding a penalty term to the ordinary cost function that discourages overly large coefficient (slope) values:

$$J_{\text{ridge}}(\theta) = J(\theta) + \lambda \cdot (\text{slope})^2$$

Here λ (**lambda**) is a hyperparameter that controls how strongly large slopes are penalized. By shrinking the coefficients, the resulting line becomes less sensitive to any single feature and less prone

to overfitting the training data. (Lasso Regression works similarly but penalizes using the absolute value of the slope rather than its square — you don't need to solve these by hand, just recognize when to reach for them.)

Rule of thumb: if a regression model shows signs of overfitting or underfitting, try Ridge or Lasso Regression instead of plain Linear Regression.

12. Hands-on Projects Referenced in This Lecture

12.1 Project 1 — Insurance Charges Prediction

Continues from the previous lecture's cleaned dataset (age, sex, BMI, children, smoker, region, etc.). A **Multiple Linear Regression** model is trained to predict **charges**. Two encoding strategies were compared:

Encoding strategy	Resulting columns	R ²	Adjusted R ²
One-Hot Encoding	34 columns	≈ 83%	≈ 83%
Label Encoding	8 columns	≈ 73%	—

Takeaway: One-Hot Encoding preserved more information (and gave a higher score) here, but at the cost of far more columns — a trade-off to keep in mind for larger datasets or when adding a user-facing form (you wouldn't want to ask a user 34 separate questions). Dimensionality-reduction techniques like PCA, or alternative models like Random Forest, help address this trade-off — covered in later lectures.

12.2 Project 2 — Ford Car Price Prediction (Kaggle)

A second, fully worked example using a Kaggle dataset (model, year, price, transmission, mileage, fuel type, tax, mpg, engine size) to predict **price**. Steps demonstrated:

- **EDA** — histogram/KDE of price distribution; correlation heatmap of numeric features against price; box plots and scatter plots comparing price against year, mileage, engine size, transmission, fuel type, and model.
- Features found to correlate meaningfully with price: **engine size, mileage, year, fuel type, and model** (transmission's relationship was judged inconclusive/"disputed").
- One-Hot Encoding of categorical columns, StandardScaler on numeric columns, 80/20 train-test split, and a LinearRegression model fit exactly as in Section 8.

13. Quick-Reference: All Key Formulas

Concept	Formula
Line equation	$y = m \cdot x + b$
Hypothesis (θ notation)	$h_{\theta}(x) = \theta_0 + \theta_1 x$
Multiple regression hypothesis	$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$
Residual error	$\text{residual} = h_{\theta}(x^{(i)}) - y^{(i)}$
Cost function (MSE)	$J(\theta_0, \theta_1) = (1/2m) \sum [h_{\theta}(x^{(i)}) - y^{(i)}]^2$
R^2	$R^2 = 1 - (\sum (y^{(i)} - \bar{y})^2) / (\sum (y^{(i)} - \hat{y}^{(i)})^2)$
Adjusted R^2	$1 - [(1 - R^2)(n - 1) / (n - p - 1)]$
Ridge penalty (concept)	$J_{\text{ridge}}(\theta) = J(\theta) + \lambda \cdot (\text{slope})^2$

14. Summary Checklist

- I can explain the difference between regression and classification.
- I understand what the best-fit line represents and how $y = mx + b$ ($h_{\theta}(x) = \theta_0 + \theta_1 x$) works.
- I can define residual error and explain why it's squared in the cost function.
- I understand the cost function $J(\theta_0, \theta_1)$ and what it means to minimize it.
- I understand how gradient descent uses the learning rate to walk toward the global minima.
- I can explain the difference between simple and multiple linear regression, and what a hyperplane is.
- I know the full workflow: split $\rightarrow$ train $\rightarrow$ predict $\rightarrow$ evaluate.
- I can interpret R^2 and explain why Adjusted R^2 is sometimes preferred.
- I can identify overfitting vs. underfitting from train/test performance gaps.
- I know Ridge/Lasso Regression are used to address over/under-fitting via a penalty term.

Next in the series: Part 3 covers **Classification algorithms** (Logistic Regression, KNN, etc.), followed by Unsupervised Learning.